

**RAJALAKSHMI ENGINEERING COLLEGE**

**An Autonomous Institution, Affiliated to Anna University**

**Rajalakshmi Nagar, Thandalam – 602 105**



**RAJALAKSHMI  
ENGINEERING  
COLLEGE**

**VII SEMESTER**

**B.Tech. – Artificial Intelligence & Data Science**

|                                  |
|----------------------------------|
| <b>CS23633 – CLOUD COMPUTING</b> |
| <b>LABORATORY RECORD</b>         |

**NAME : BIJLESH S**

**REGISTER NO: 2116231801501**

**ACADEMIC YEAR: 2026-2027**



**RAJALAKSHMI  
ENGINEERING  
COLLEGE**

**BONAFIDE CERTIFICATE**

**NAME** BIJLESH S

**ACADEMIC YEAR** 2026-2027 **SEMESTER** VII

**BRANCH** B.Tech - Artificial Intelligence and Data Science

**UNIVERSITY REGISTER No.**

**2116231801501**

Certified that this is the bonafide record of work done by the above student in the  
**Cloud Computing** Laboratory in the year **2026 - 2027**

**Signature of Faculty - in - Charge**

**Submitted for the Practical Examination**

**held on .....**

**Internal Examiner**

**External Examiner**

## INDEX

| S.No | Name of the Experiment                                         | Date       | Github QR                                                                             | Faculty Sign |
|------|----------------------------------------------------------------|------------|---------------------------------------------------------------------------------------|--------------|
| 1    | Private Cloud Using Openstack                                  | 24.06.2026 |    |              |
| 2    | PaaS in Public Cloud Using Render                              | 08.07.2026 |    |              |
| 3    | Setting Up Hybrid Cloud Connectivity Using Cisco Packet Tracer | 22.07.2026 |   |              |
| 4    | Virtual Private Server (VPS) Provisioning and Configuration    | 05.08.2026 |  |              |
| 5    | Cloud Backup and Migration using Google Drive                  | 14.08.2026 |  |              |
| 6    | CDN Service Selection and Configuration                        | 21.08.2026 |  |              |

|                                   |                                      |
|-----------------------------------|--------------------------------------|
| <b>Exp No:1</b>                   | <b>Private Cloud using OpenStack</b> |
| <b>Date:</b><br><b>24/06/2026</b> |                                      |

**Aim:**

To create a private cloud environment by installing and configuring OpenStack (MicroStack) on a single Ubuntu system. The experiment also helps in learning how cloud resources such as virtual machines, networking, and storage are managed in a private cloud.

**What is Openstack?**

OpenStack is an open-source cloud platform used to build and manage cloud infrastructure. It is mainly used for Infrastructure as a Service (IaaS), where users can create and control virtual machines whenever required. It provides services for computing, networking, and storage in a cloud environment.

**What is Microstack?**

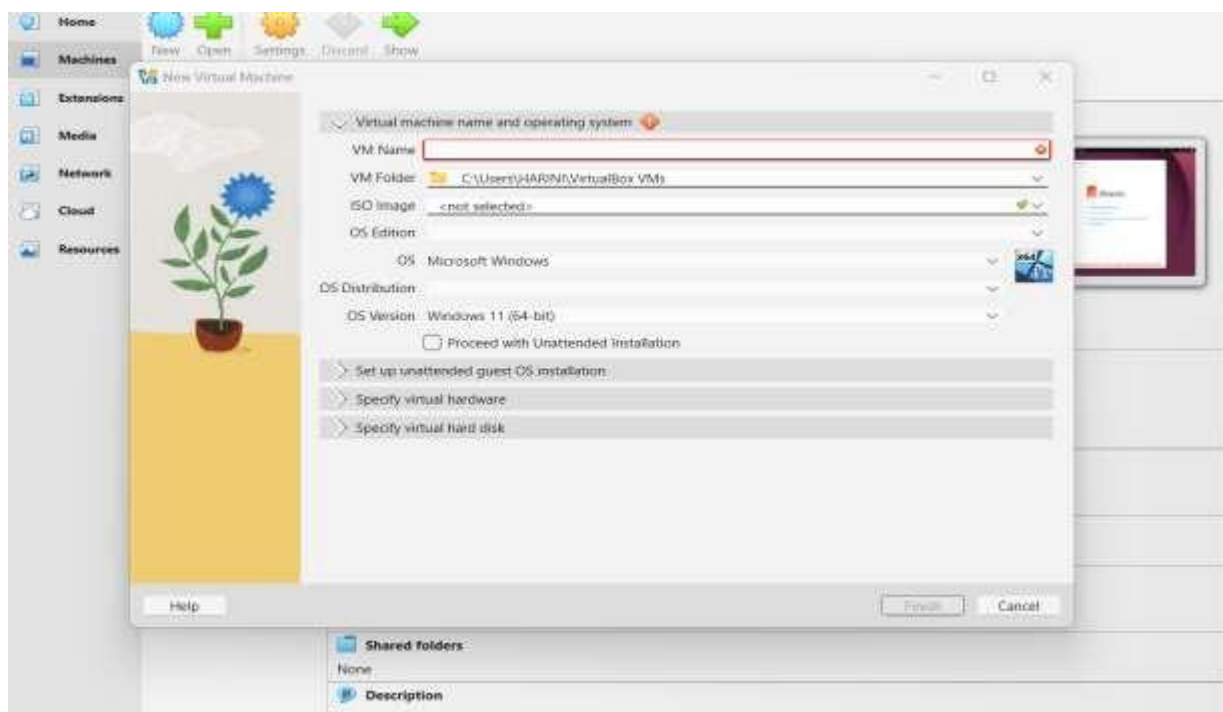
MicroStack is a lightweight version of OpenStack that runs on a single computer. It is mainly used for learning, testing, and small-scale development. Since it requires only minimal configuration, it is a good choice for beginners who want to understand how OpenStack works.

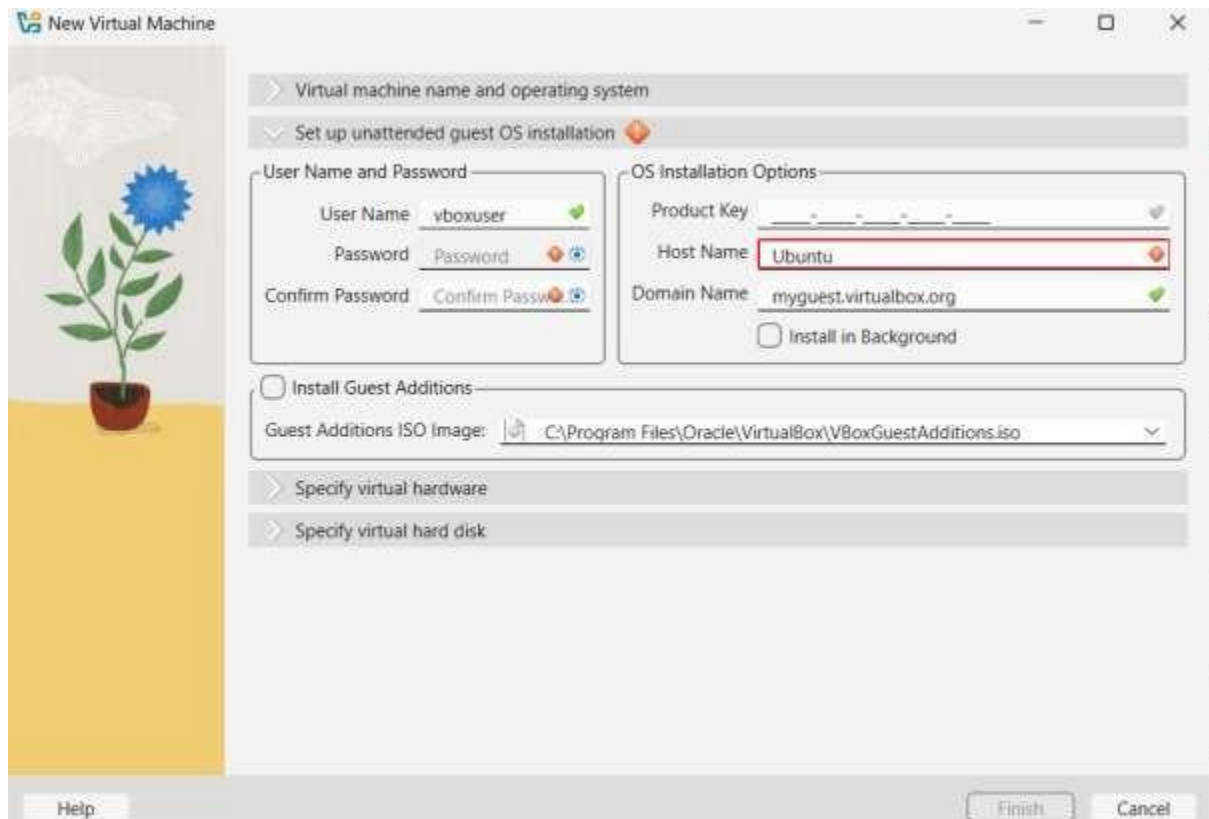
## Procedure:

### Step 1

Create a new Ubuntu virtual machine using Oracle VirtualBox with the following configuration:

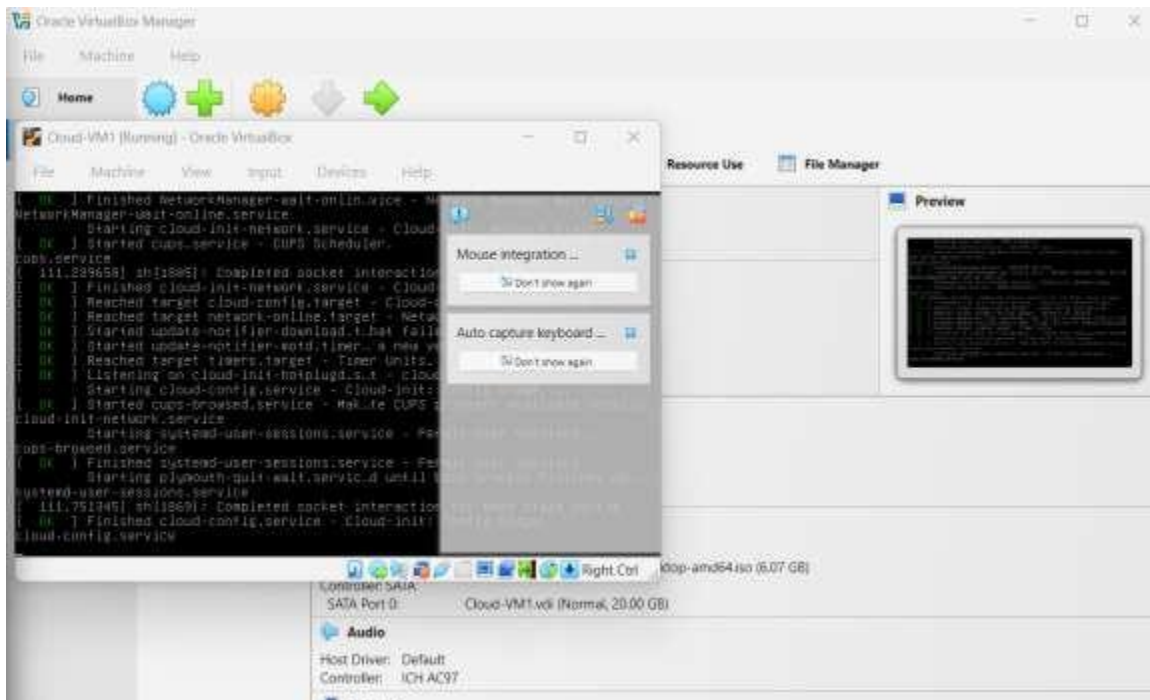
- **Memory:** 8 GB RAM
- **Processor:** 4 CPU Cores
- **Storage:** 20 GB Virtual Disk





## Step 2

Install the Ubuntu operating system and log in using the username and password created during installation.



### Step 3

Update the Ubuntu packages: `sudo apt update`  
`sudo apt upgrade`

### Step 4

Install MicroStack (Beta channel): `sudo snap install microstack --beta`

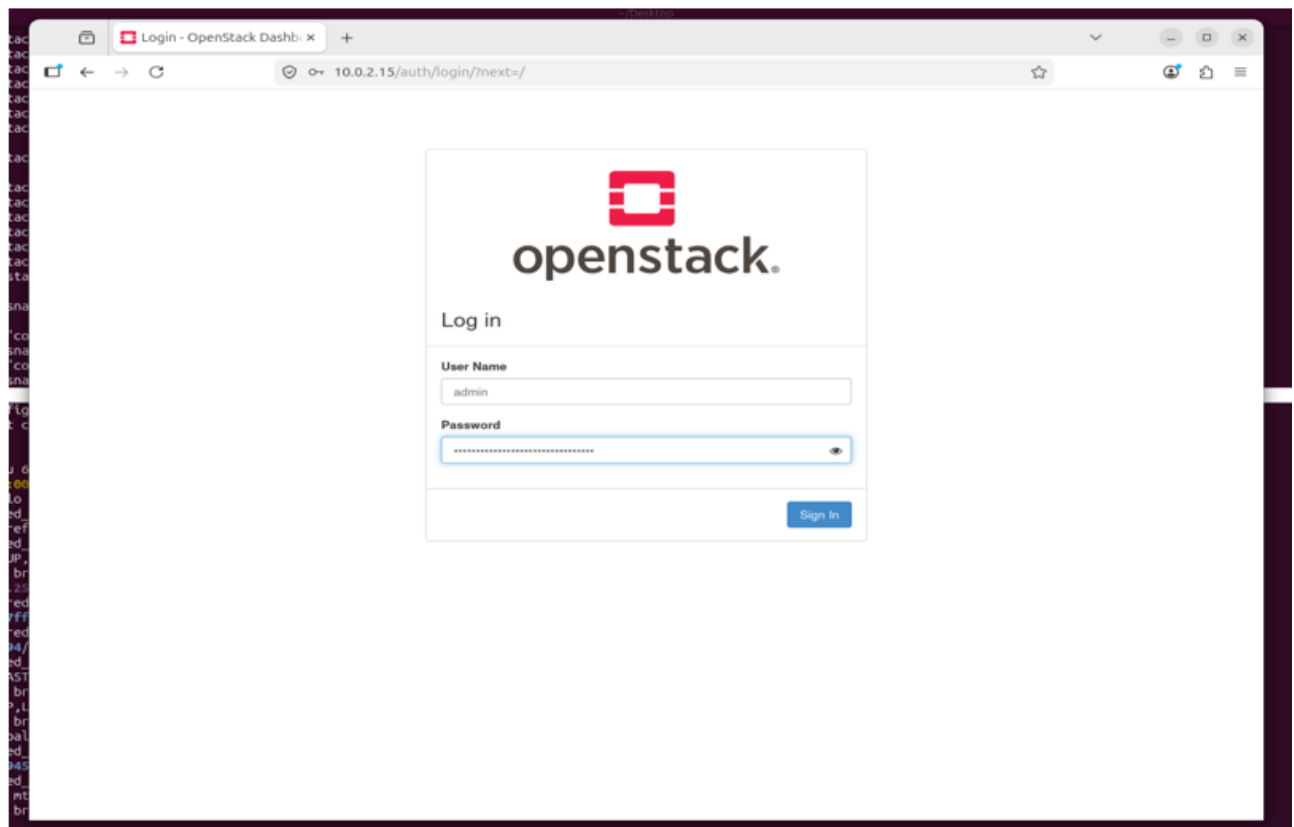
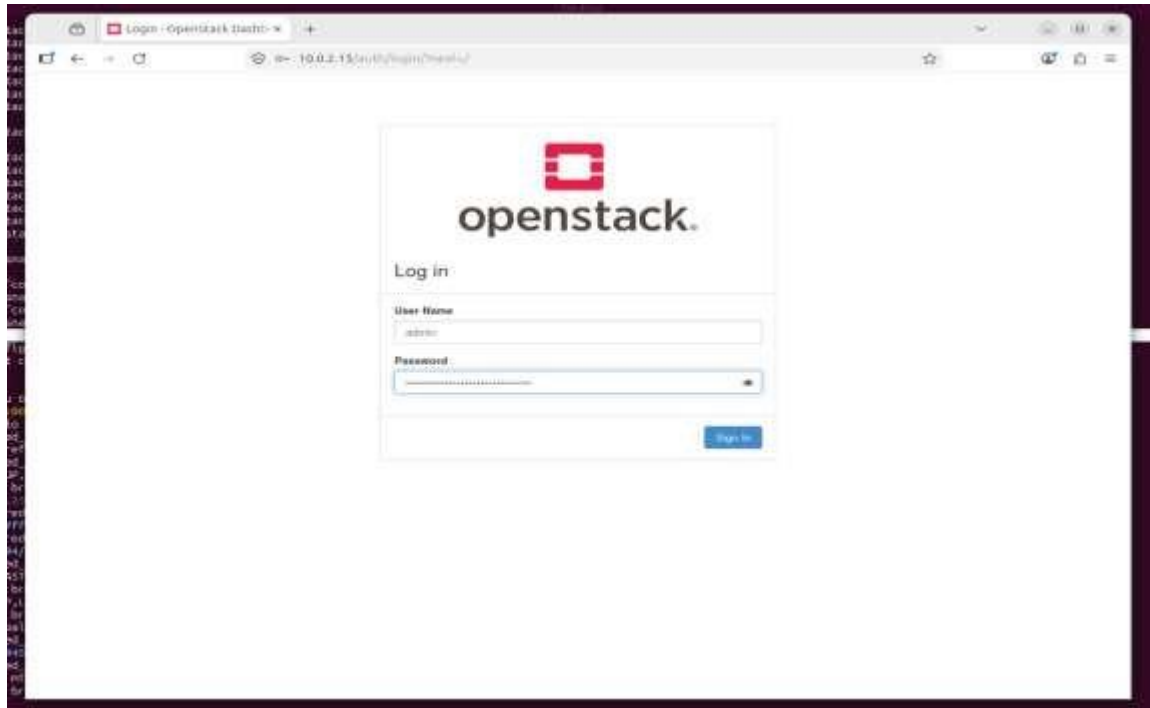
### Step 5

Check whether MicroStack has been installed successfully. `snap list microstack`

### Step 6

Initialize the OpenStack services automatically by executing the following

command. `sudo microstack init --auto --control`



## Step 7

Verify the installed OpenStack version:

```
microstack.openstack --version
```

## Step 8

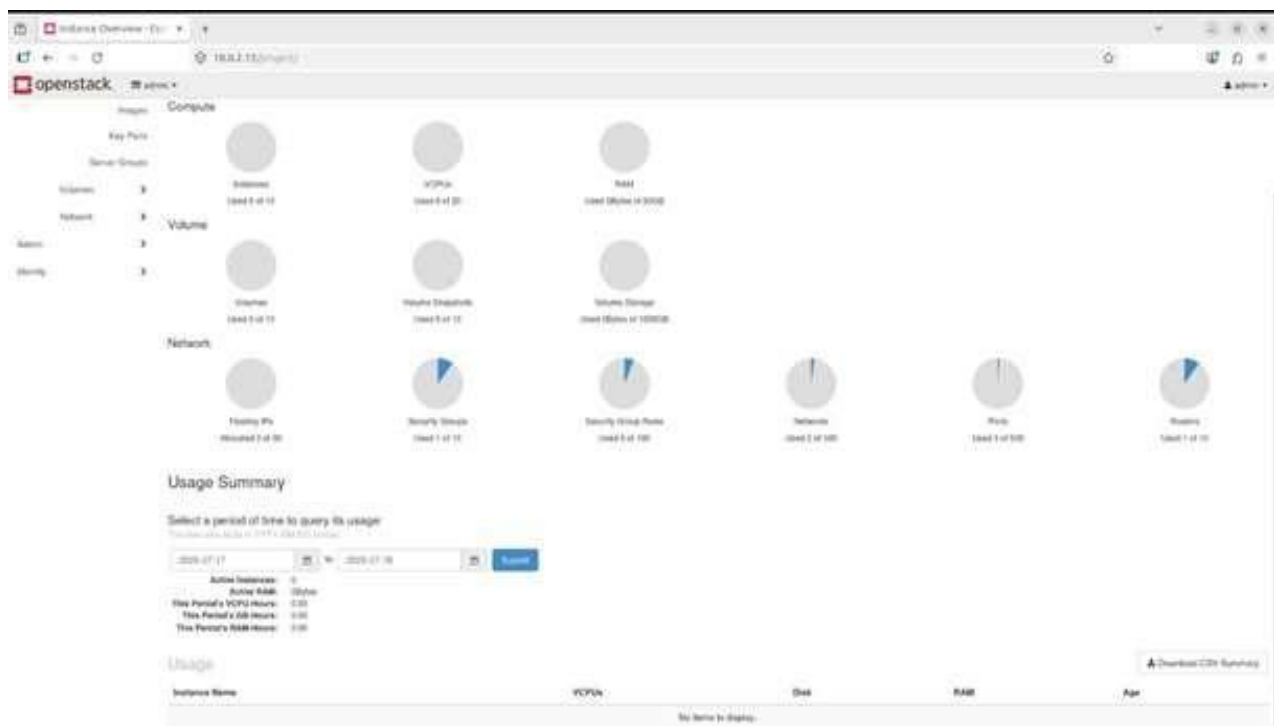
Retrieve the Keystone administrator password:

```
sudo snap get microstack config.credentials.keystone-password
```

## Step 9

Open the OpenStack Horizon Dashboard in the web browser using the IP address provided after initialization and log in using:

- Username: admin
- Password: (obtained from the previous command)



**Result:**

OpenStack (MicroStack) was successfully installed and configured on Ubuntu, and the Horizon Dashboard was accessed s

|                                   |                                                           |
|-----------------------------------|-----------------------------------------------------------|
| <b>Exp No:2</b>                   | <b>Public Cloud Application<br/>Deployment Using PaaS</b> |
| <b>Date:</b><br><b>08/07/2026</b> |                                                           |

**Aim:**

To develop and deploy a simple web application using Render, a Platform as a Service (PaaS), and understand how cloud platforms host applications without requiring users to manage servers.

**Objective:**

- To understand the concept of **Platform as a Service (PaaS)**.
- To create a simple web application using the **Flask** framework.
- To deploy the Flask application on the **Render** cloud platform.
- To access the deployed application through a public URL and verify its functionality.

**Theory:**

**Platform as a Service (PaaS)** is a cloud computing model that provides a complete platform for developing, deploying, and managing applications without requiring users to manage the underlying infrastructure. The cloud service provider is responsible for managing the servers, operating system, networking, software installation, and maintenance, while developers only need to write the application code.

In this experiment, **Render** is used as the PaaS platform. Render automatically creates the server environment, installs the required Python packages, deploys the Flask application, provides a public URL, and manages the application without requiring manual server configuration. This simplifies application deployment and allows developers to focus on building their applications rather than managing infrastructure.

**Software Requirements:**

- Operating System: Windows 10/11 or Linux
- Python 3.10 or later
- Flask Framework
- Gunicorn
- Git
- GitHub Account
- Render Account
- Visual Studio Code (or any Python IDE)
- Web Browser (Google Chrome/Microsoft Edge)
- Internet Connection

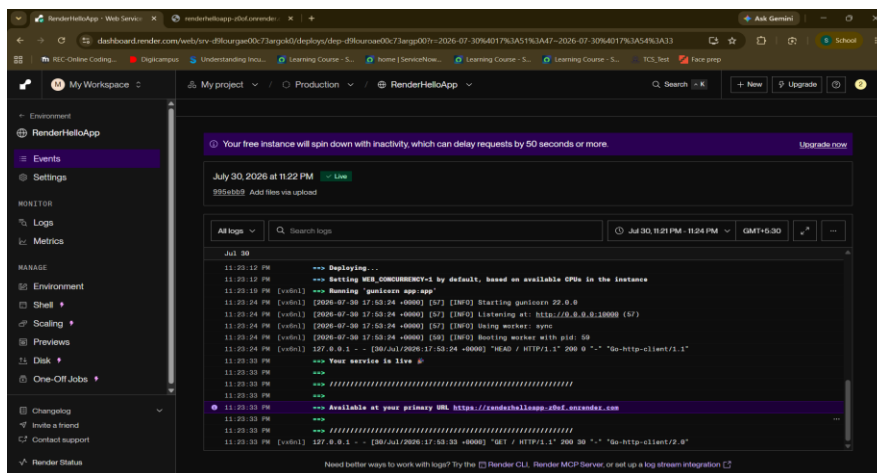
**Procedure:**

1. Installed the required software, including Python, Git, and Visual Studio Code. Created GitHub and Render accounts.
2. Created a project folder named **RenderHelloApp** and added the files `app.py` and `requirements.txt`.
3. Developed a Flask web application in `app.py` with two routes:
  - `/` – displays **"Hello! Welcome to Render PaaS."**
  - `/about` – displays **"This application is deployed on Render."**
4. Added the required Python dependencies (Flask and Gunicorn) in the `requirements.txt` file.
5. Tested the application locally using:

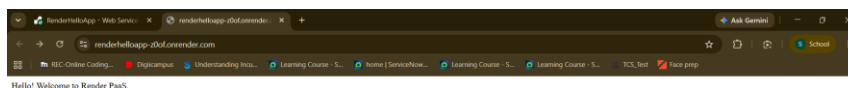
```
python app.py
```

Verified that both the Home page and About page worked correctly in the browser.

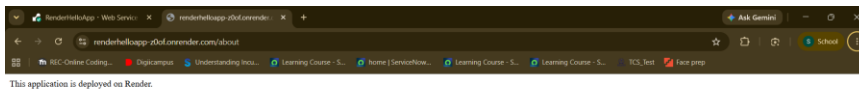
6. Created a public GitHub repository named **RenderHelloApp** and uploaded the project files.
7. Logged in to Render, created a **Web Service**, selected the GitHub repository, and configured the deployment with:
  - Runtime: Python
  - Build Command: `pip install -r requirements.txt`
  - Start Command: `gunicorn app:app`
8. Deployed the application successfully on Render and obtained a public URL.



Render Dashboard showing "**Live**" deployment.



Home page displaying "**Hello! Welcome to Render PaaS.**"



---

About page displaying **"This application is deployed on Render."**

### **Result:**

The Flask web application was successfully developed and deployed on the **Render** cloud platform. The application was accessible through a public URL, and both the **Home** and **About** pages functioned correctly. This experiment demonstrated the use of **Platform as a Service (PaaS)** for deploying web applications without managing the underlying infrastructure.

|                                   |                                                                                                             |
|-----------------------------------|-------------------------------------------------------------------------------------------------------------|
| <b>Exp No:3</b>                   | <h2 style="text-align: center;">Setting up Hybrid Cloud<br/>Connectivity using Cisco Packet<br/>Tracer</h2> |
| <b>Date:</b><br><b>22/07/2026</b> |                                                                                                             |

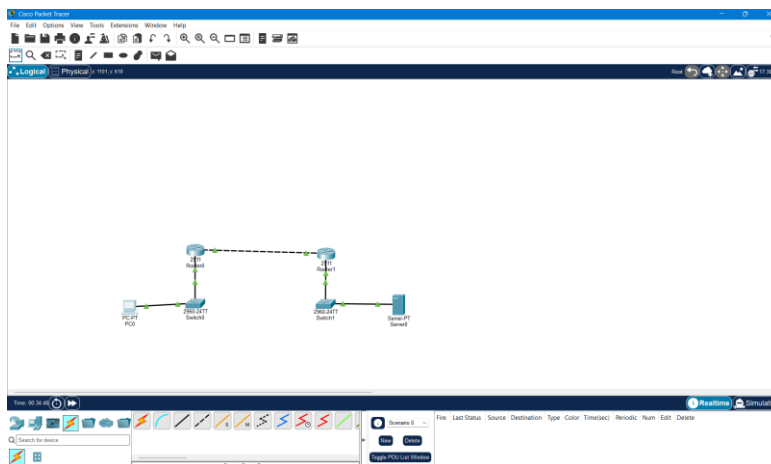
### Aim:

To simulate a secure hybrid cloud connection between an on-premises data center and a cloud network using routers, switches, subnets, and routing in Cisco Packet Tracer.

### Software Used:

- Cisco Packet Tracer
- Windows 11 (or your OS)

### Network Topology:



Hybrid cloud network topology.

- PC
- Switch0
- Router0

- Router1
- Switch1
- Server
- All green links

## Router0 Configuration

```

Router0
Physical Config CLI Attributes
IOS Command Line Interface

Router(config-if)#
Router(config-if)#no shutdown

Router(config-if)#
%LINK-5-CHANGED: Interface GigabitEthernet0/0, changed state to up

%LINEPROTO-5-UPDOWN: Line protocol on Interface GigabitEthernet0/0, changed state to up
exit
Router(config)#interface gigabitEthernet0/1
Router(config-if)#ip address 10.10.10.1 255.255.255.252
Router(config-if)#no shutdown

Router(config-if)#
%LINK-5-CHANGED: Interface GigabitEthernet0/1, changed state to up
exit
Router(config)#do show ip interface brief
Interface      IP-Address      OK? Method Status      Protocol
GigabitEthernet0/0  192.168.1.1    YES manual up          up
GigabitEthernet0/1  10.10.10.1     YES manual up          down
GigabitEthernet0/2  unassigned     YES unset  administratively down down
Vlan1          unassigned     YES unset  administratively down down
Router(config)#
Router#
%SYS-5-CONFIG_I: Configured from console by console

Router#
%LINEPROTO-5-UPDOWN: Line protocol on Interface GigabitEthernet0/1, changed state to up

Router#show ip interface brief
Interface      IP-Address      OK? Method Status      Protocol
GigabitEthernet0/0  192.168.1.1    YES manual up          up
GigabitEthernet0/1  10.10.10.1     YES manual up          up
GigabitEthernet0/2  unassigned     YES unset  administratively down down
Vlan1          unassigned     YES unset  administratively down down
Router#configure terminal
Enter configuration commands, one per line. End with CNTL/Z.
Router(config)#ip route 10.0.0.0 255.255.255.0 10.10.10.2
Router(config)#exit
Router#
%SYS-5-CONFIG_I: Configured from console by console
copy running-config startup-config
Destination filename [startup-config]?
Building configuration...
[OK]
Router#

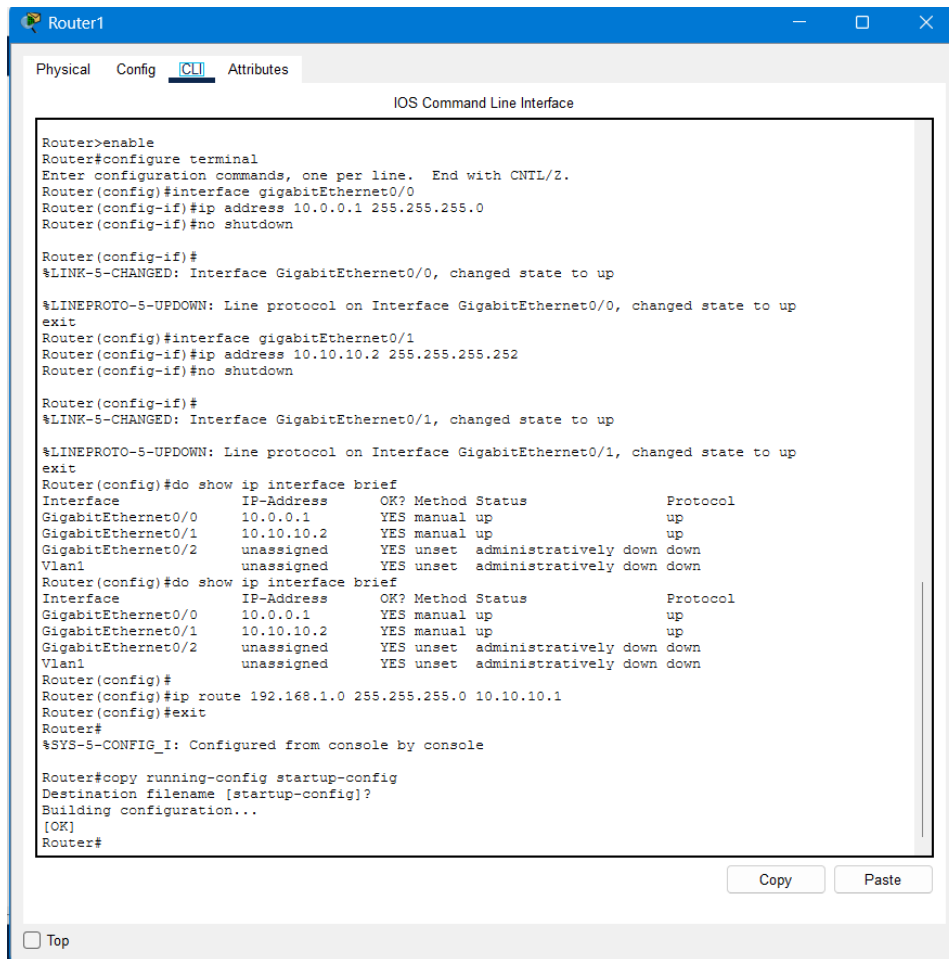
```

Copy Paste

☐ Top

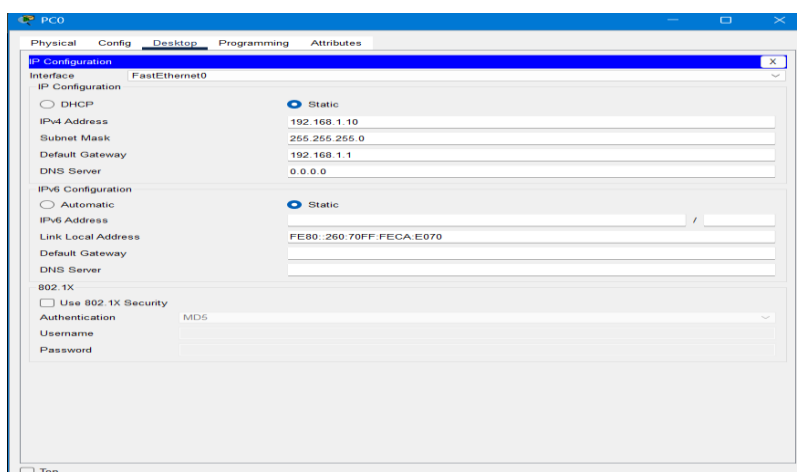
Router0 interface configuration.

## Router1 Configuration:



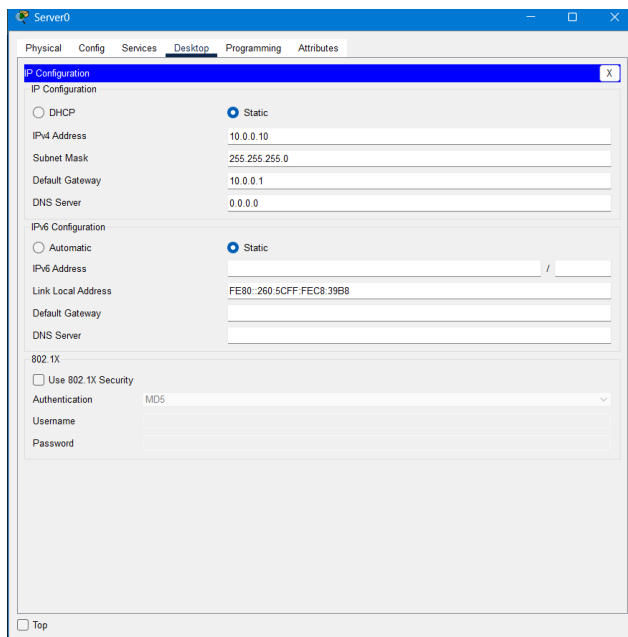
Router1 interface configuration.

## PC Configuration:



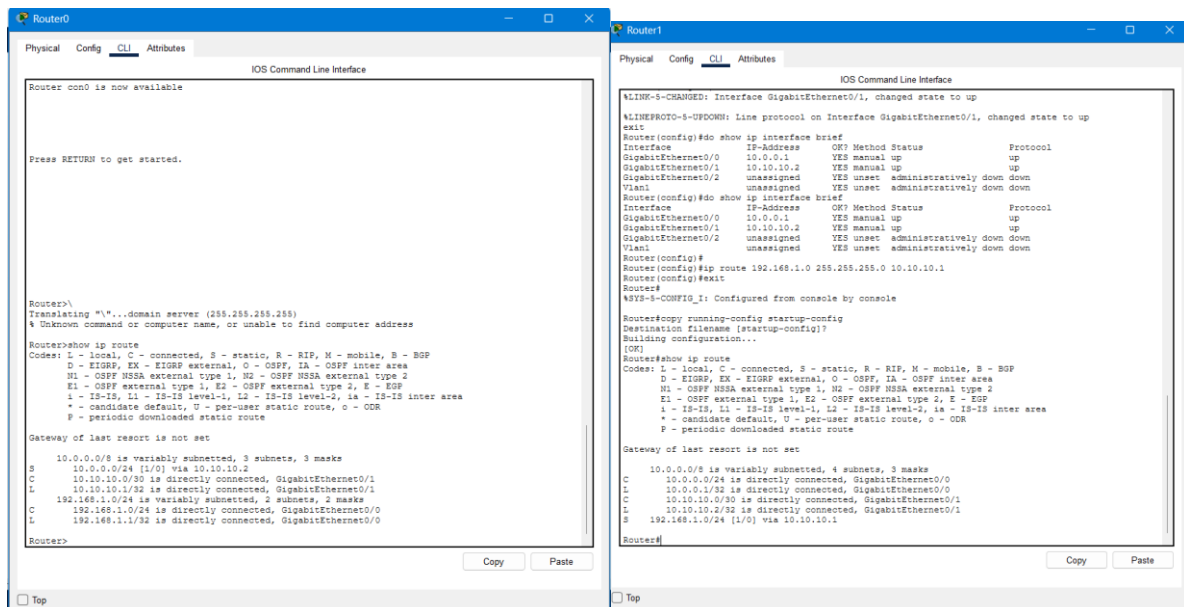
PC IP configuration.

## Server Configuration:



Server IP configuration.

## Static Route Configuration:



Static routing configuration.

## Connectivity Test:

```
Cisco Packet Tracer PC Command Line 1.0
C:\>
C:\>ping 10.0.0.10

Pinging 10.0.0.10 with 32 bytes of data:

Request timed out.
Request timed out.
Reply from 10.0.0.10: bytes=32 time=1ms TTL=126
Reply from 10.0.0.10: bytes=32 time=1ms TTL=126

Ping statistics for 10.0.0.10:
    Packets: Sent = 4, Received = 2, Lost = 2 (50% loss),
    Approximate round trip times in milli-seconds:
        Minimum = 0ms, Maximum = 1ms, Average = 5ms
C:\>ping 10.0.0.10

Pinging 10.0.0.10 with 32 bytes of data:

Reply from 10.0.0.10: bytes=32 time=1ms TTL=126
Reply from 10.0.0.10: bytes=32 time=1ms TTL=126
Reply from 10.0.0.10: bytes=32 time=1ms TTL=126
Reply from 10.0.0.10: bytes=32 time=1ms TTL=126

Ping statistics for 10.0.0.10:
    Packets: Sent = 4, Received = 4, Lost = 0 (0% loss),
    Approximate round trip times in milli-seconds:
        Minimum = 0ms, Maximum = 0ms, Average = 0ms
C:\>
```

Successful ping from the PC to the cloud server.

## Result:

The hybrid cloud network was successfully simulated using Cisco Packet Tracer. Two routers were configured with IP addressing and static routing to interconnect the on-premises network and the cloud network. End-to-end communication between the PC and the server was verified successfully using the ping command, confirming successful hybrid cloud connectivity.

|                                   |                                                         |
|-----------------------------------|---------------------------------------------------------|
| <b>Exp No:4</b>                   | <b>Basic VPS Server Setup and Network Configuration</b> |
| <b>Date:</b><br><b>05/08/2026</b> |                                                         |

**Aim:**

To provision a Virtual Private Server (VPS) using VirtualBox, install Ubuntu Server, configure network settings (IP Address, Subnet Mask, Gateway, DNS), and enable SSH remote access.

**What is VPS?**

A Virtual Private Server (VPS) is a software-based server created using virtualization technology. It operates independently with its own operating system, storage, memory, and network configuration while sharing the physical hardware with other virtual machines. Virtualization allows efficient utilization of resources and forms the foundation of cloud computing platforms.

Network configuration parameters include:

**IP Address**

A unique address assigned to a device on a network.

**Subnet Mask**

Determines the network and host portions of an IP address.

**Gateway**

Provides access to networks outside the local network.

**DNS (Domain Name System)**

Converts domain names into IP addresses.

**SSH (Secure Shell)**

A secure protocol used for remote administration of Linux systems.

## PROCEDURE:

1. Open the Ubuntu terminal and verify the internet connection using the ping google.com command.

```
valid_lft forever preferred_lft forever
vboxuser@Ubuntu:~/Desktop$ ping google.com
PING google.com (142.251.221.206) 56(84) bytes of data.
64 bytes from pnmaaa-ba-in-f14.1e100.net (142.251.221.206): icmp_seq=1 ttl=64 time=35.8 ms
64 bytes from pnmaaa-ba-in-f14.1e100.net (142.251.221.206): icmp_seq=2 ttl=64 time=36.1 ms
64 bytes from pnmaaa-ba-in-f14.1e100.net (142.251.221.206): icmp_seq=3 ttl=64 time=64.2 ms
64 bytes from pnmaaa-ba-in-f14.1e100.net (142.251.221.206): icmp_seq=4 ttl=64 time=51.4 ms
64 bytes from pnmaaa-ba-in-f14.1e100.net (142.251.221.206): icmp_seq=5 ttl=64 time=60.5 ms
64 bytes from pnmaaa-ba-in-f14.1e100.net (142.251.221.206): icmp_seq=6 ttl=64 time=239 ms
64 bytes from pnmaaa-ba-in-f14.1e100.net (142.251.221.206): icmp_seq=7 ttl=64 time=256 ms
64 bytes from pnmaaa-ba-in-f14.1e100.net (142.251.221.206): icmp_seq=8 ttl=64 time=93.1 ms
64 bytes from pnmaaa-ba-in-f14.1e100.net (142.251.221.206): icmp_seq=9 ttl=64 time=48.6 ms
64 bytes from pnmaaa-ba-in-f14.1e100.net (142.251.221.206): icmp_seq=10 ttl=64 time=34.1 ms
64 bytes from pnmaaa-ba-in-f14.1e100.net (142.251.221.206): icmp_seq=11 ttl=64 time=55.8 ms
64 bytes from pnmaaa-ba-in-f14.1e100.net (142.251.221.206): icmp_seq=12 ttl=64 time=51.9 ms
64 bytes from pnmaaa-ba-in-f14.1e100.net (142.251.221.206): icmp_seq=13 ttl=64 time=33.4 ms
64 bytes from pnmaaa-ba-in-f14.1e100.net (142.251.221.206): icmp_seq=14 ttl=64 time=149 ms
64 bytes from pnmaaa-ba-in-f14.1e100.net (142.251.221.206): icmp_seq=15 ttl=64 time=246 ms
64 bytes from pnmaaa-ba-in-f14.1e100.net (142.251.221.206): icmp_seq=16 ttl=64 time=64.7 ms
64 bytes from pnmaaa-ba-in-f14.1e100.net (142.251.221.206): icmp_seq=17 ttl=64 time=81.8 ms
64 bytes from pnmaaa-ba-in-f14.1e100.net (142.251.221.206): icmp_seq=18 ttl=64 time=48.2 ms
64 bytes from pnmaaa-ba-in-f14.1e100.net (142.251.221.206): icmp_seq=19 ttl=64 time=44.5 ms
64 bytes from pnmaaa-ba-in-f14.1e100.net (142.251.221.206): icmp_seq=20 ttl=64 time=264 ms
64 bytes from pnmaaa-ba-in-f14.1e100.net (142.251.221.206): icmp_seq=21 ttl=64 time=122 ms
64 bytes from pnmaaa-ba-in-f14.1e100.net (142.251.221.206): icmp_seq=22 ttl=64 time=50.8 ms
64 bytes from pnmaaa-ba-in-f14.1e100.net (142.251.221.206): icmp_seq=23 ttl=64 time=48.9 ms
64 bytes from pnmaaa-ba-in-f14.1e100.net (142.251.221.206): icmp_seq=24 ttl=64 time=52.6 ms
64 bytes from pnmaaa-ba-in-f14.1e100.net (142.251.221.206): icmp_seq=25 ttl=64 time=34.0 ms
64 bytes from pnmaaa-ba-in-f14.1e100.net (142.251.221.206): icmp_seq=26 ttl=64 time=56.1 ms
64 bytes from pnmaaa-ba-in-f14.1e100.net (142.251.221.206): icmp_seq=27 ttl=64 time=60.1 ms
64 bytes from pnmaaa-ba-in-f14.1e100.net (142.251.221.206): icmp_seq=28 ttl=64 time=148 ms
64 bytes from pnmaaa-ba-in-f14.1e100.net (142.251.221.206): icmp_seq=29 ttl=64 time=45.3 ms
64 bytes from pnmaaa-ba-in-f14.1e100.net (142.251.221.206): icmp_seq=30 ttl=64 time=64.6 ms
64 bytes from pnmaaa-ba-in-f14.1e100.net (142.251.221.206): icmp_seq=31 ttl=64 time=277 ms
64 bytes from pnmaaa-ba-in-f14.1e100.net (142.251.221.206): icmp_seq=32 ttl=64 time=42.7 ms
64 bytes from pnmaaa-ba-in-f14.1e100.net (142.251.221.206): icmp_seq=33 ttl=64 time=36.0 ms
64 bytes from pnmaaa-ba-in-f14.1e100.net (142.251.221.206): icmp_seq=34 ttl=64 time=241 ms
64 bytes from pnmaaa-ba-in-f14.1e100.net (142.251.221.206): icmp_seq=35 ttl=64 time=301 ms
64 bytes from pnmaaa-ba-in-f14.1e100.net (142.251.221.206): icmp_seq=36 ttl=64 time=37.5 ms
64 bytes from pnmaaa-ba-in-f14.1e100.net (142.251.221.206): icmp_seq=37 ttl=64 time=50.2 ms
```

2. Check the network routing information using the ip route command.

```
valid_lft forever preferred_lft forever
vboxuser@Ubuntu:~/Desktop$ ip route
default via 10.0.2.2 dev enp0s3 proto dhcp src 10.0.2.15 metric 100
10.0.2.0/24 dev enp0s3 proto kernel scope link src 10.0.2.15 metric 100
10.20.20.0/24 dev br-ex proto kernel scope link src 10.20.20.1
```

3. Verify the DNS (Domain Name System) configuration of the system using the cat /etc/resolv.conf command.

```
vboxuser@Ubuntu:~/Desktop$ cat /etc/resolv.conf
# This is /run/systemd/resolve/stub-resolv.conf managed by man:systemd-resolved(8).
# Do not edit.
#
# This file might be symlinked as /etc/resolv.conf. If you're looking at
# /etc/resolv.conf and seeing this text, you have followed the symlink.
#
# This is a dynamic resolv.conf file for connecting local clients to the
# internal DNS stub resolver of systemd-resolved. This file lists all
# configured search domains.
#
# Run "resolvectl status" to see details about the uplink DNS servers
# currently in use.
#
# Third party programs should typically not access this file directly, but only
# through the symlink at /etc/resolv.conf. To manage man:resolv.conf(5) in a
# different way, replace this symlink by a static file or a different symlink.
#
# See man:systemd-resolved.service(8) for details about the supported modes of
# operation for /etc/resolv.conf.

nameserver 127.0.0.53
options edns0 trust-ad
search .
```

4. Update the Ubuntu package list (if required) using: `sudo apt update`
5. Install the OpenSSH Server package using: `sudo apt install openssh-server -y`
6. Enable and start the SSH service using:
7. `sudo systemctl enable ssh`  
`sudo systemctl start ssh`
8. Check whether the SSH service is running successfully using: `sudo systemctl status`

```
Processing triggers for dmw (0.30.2-900001)
vboxuser@Ubuntu:~/Desktop$ sudo systemctl status ssh
○ ssh.service - OpenBSD Secure Shell server
   Loaded: loaded (/usr/lib/systemd/system/ssh.service; disabled; preset: enabled)
   Active: inactive (dead)
   TriggeredBy: ● ssh.socket
   Docs: man:sshd(8)
        man:sshd_config(5)
```

9. Find the IP address of the Ubuntu system using: `Hostname -I`
10. Establish an SSH connection to the Ubuntu

system using: ssh <username>@<IP\_address>

```

vboxuser@Ubuntu:~/Desktop$ hostname -I
10.0.2.15 10.20.20.1 fd17:625c:f037:2:a00:27ff:fe5c:c094
vboxuser@Ubuntu:~/Desktop$ ssh vboxuser@10.0.2.15
The authenticity of host '10.0.2.15 (10.0.2.15)' can't be established.
ED25519 key fingerprint is: SHA256:DPutENGud479D6pbIQjtRgFAhR1uP74ZPyEInGKGpBA
This key is not known by any other names.
Are you sure you want to continue connecting (yes/no/[fingerprint])? yes
Warning: Permanently added '10.0.2.15' (ED25519) to the list of known hosts.
vboxuser@10.0.2.15's password:
Welcome to Ubuntu 26.04 LTS (GNU/Linux 7.0.0-28-generic x86_64)

 * Documentation:  https://docs.ubuntu.com
 * Management:    https://landscape.canonical.com
 * Support:       https://ubuntu.com/pro

Expanded Security Maintenance for Applications is not enabled.

0 updates can be applied immediately.

Enable ESM Apps to receive additional future security updates.
See https://ubuntu.com/esm or run: sudo pro status

*** System restart required ***

The programs included with the Ubuntu system are free software;
the exact distribution terms for each program are described in the
individual files in /usr/share/doc/*/copyright.

Ubuntu comes with ABSOLUTELY NO WARRANTY, to the extent permitted by
applicable law.

```

11. Verify the successful SSH login by checking the "Welcome to Ubuntu" message displayed on the terminal.

### Result:

The Ubuntu system was successfully configured and verified for network connectivity. The OpenSSH Server was installed and enabled, and the SSH service was confirmed to be running. The system's IP address was obtained, and a successful SSH login was performed. Thus, secure remote access to the Ubuntu system was successfully established and verified.

|                                   |                                                             |
|-----------------------------------|-------------------------------------------------------------|
| <b>Exp No:5</b>                   | <b>Cloud Migration and Backup Setup Using<br/>Amazon S3</b> |
| <b>Date:</b><br><b>14/08/2026</b> |                                                             |

**Aim:**

To create a cloud backup system by storing important files in Google Drive, scheduling automatic backups, and verifying successful backup completion.

**Objective:**

- To understand the concept of cloud backup and cloud migration.
- To create sample data files and store them in Google Drive.
- To install Google Drive for Desktop and enable folder synchronization.
- To modify a local file and verify that the change is automatically backed up to the cloud.

**Theory:**

Cloud Backup means storing a copy of important files on cloud servers instead of keeping them only on a local computer. This provides a safe copy of the data in the cloud so that the files can still be accessed if the local computer fails.

Cloud Migration means moving files, applications, or data from a local computer to a cloud service. In this experiment, Google Drive is used as the cloud storage service. The local OfficeData folder is synchronized with Google Drive using Google Drive for Desktop, so that changes made to the local files are automatically reflected in the cloud.

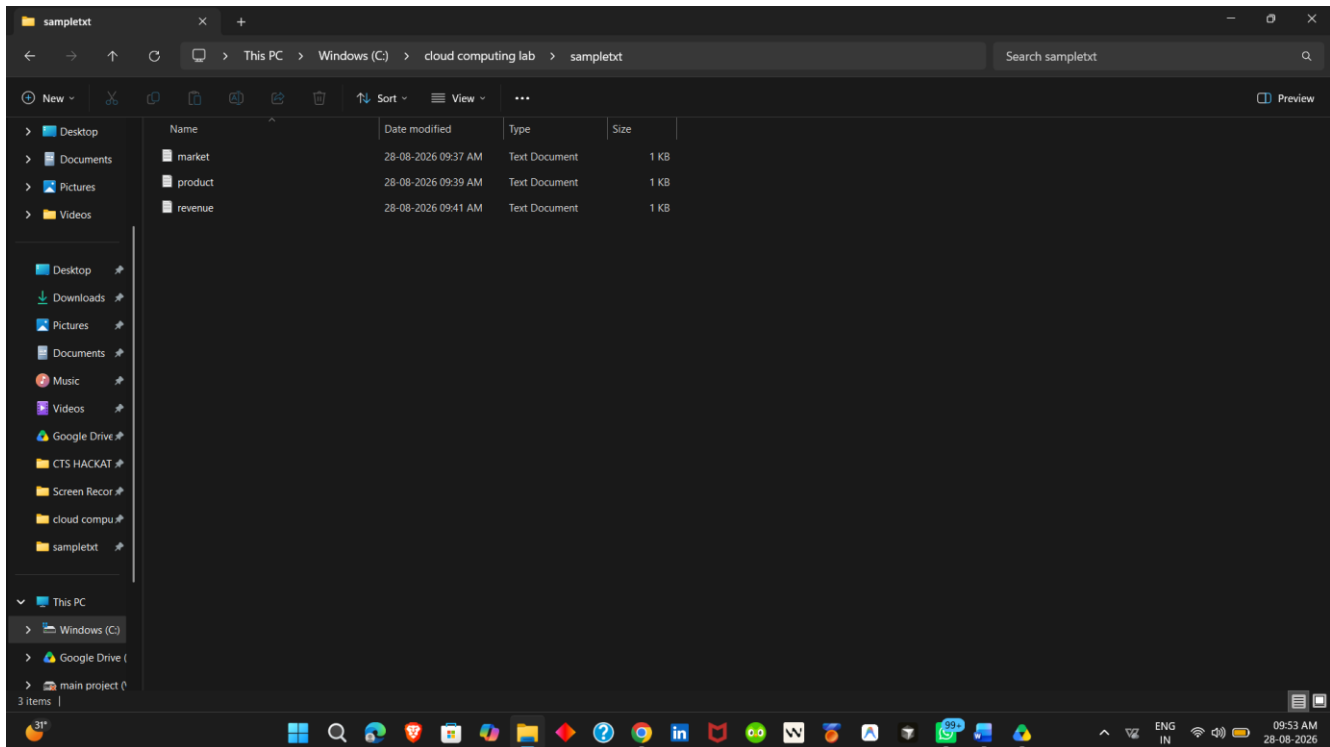
**Software Requirements:**

- Operating System: Windows 10/11
- Google Account
- Google Drive / Google Drive for Desktop
- Web Browser (Google Chrome/Microsoft Edge)
- Internet Connection

**Procedure:**

1. Created a folder named OfficeData on the Desktop.

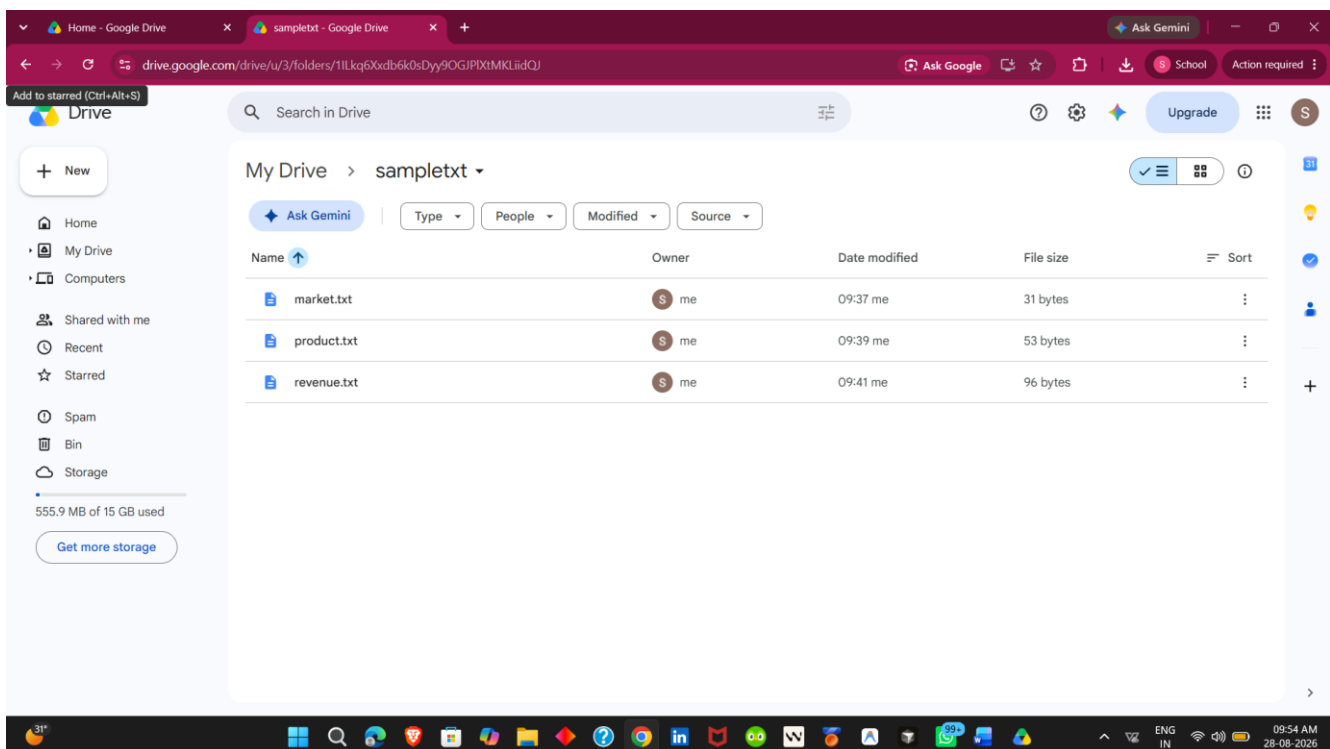
2. Created three text files inside OfficeData: Student.txt, Attendance.txt, and Marks.txt, and added sample content to each file.



3. Opened Google Drive in the browser and logged in using the Google account.

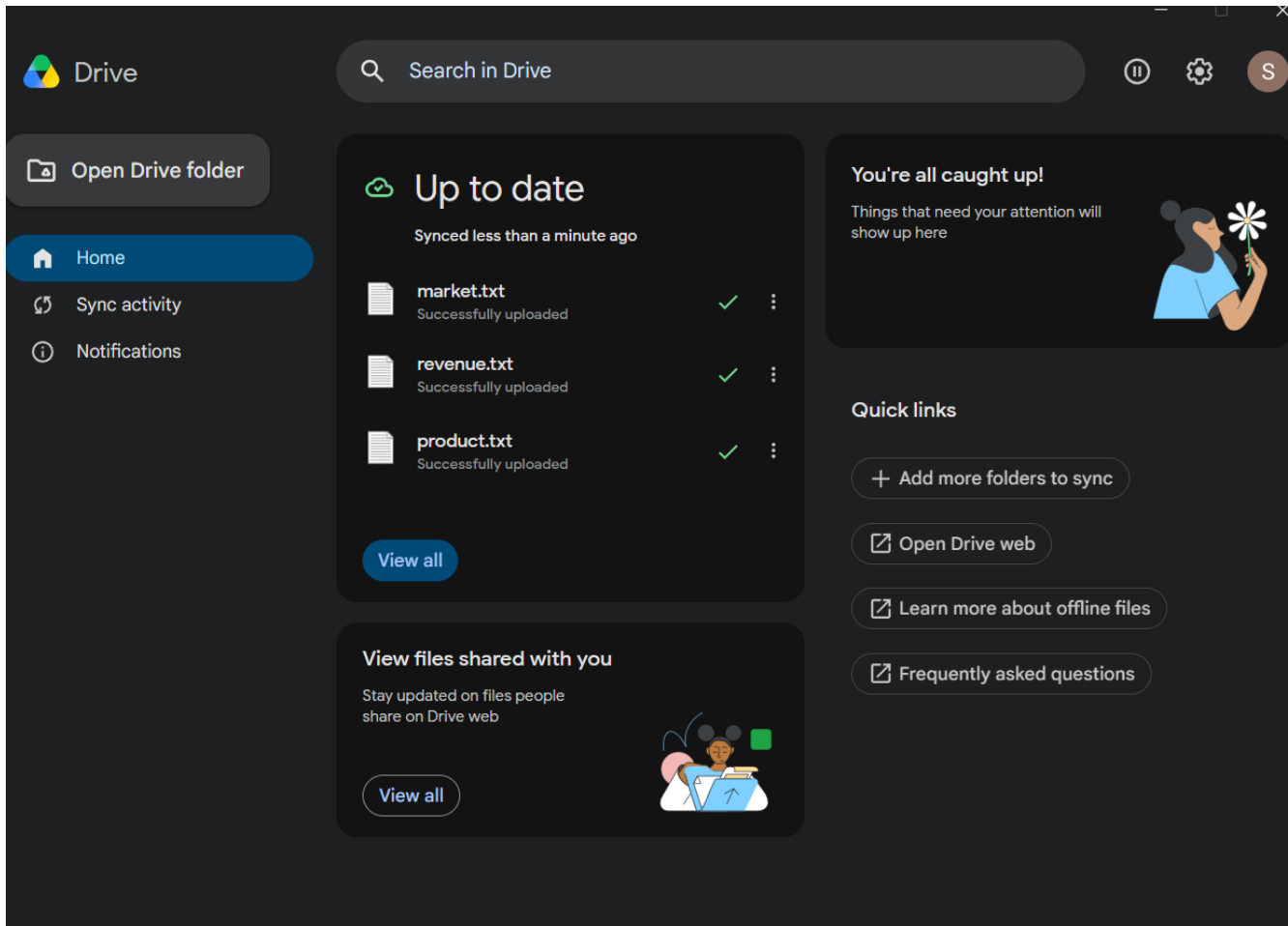
4. Created a folder named CloudBackup in Google Drive.

5. Opened CloudBackup and uploaded Student.txt, Attendance.txt, and Marks.txt.



6. Downloaded and installed Google Drive for Desktop and signed in using the same Google account.

7. Opened Google Drive Settings, added the Desktop\OfficeData folder, enabled Sync with Google Drive, and completed the setup.

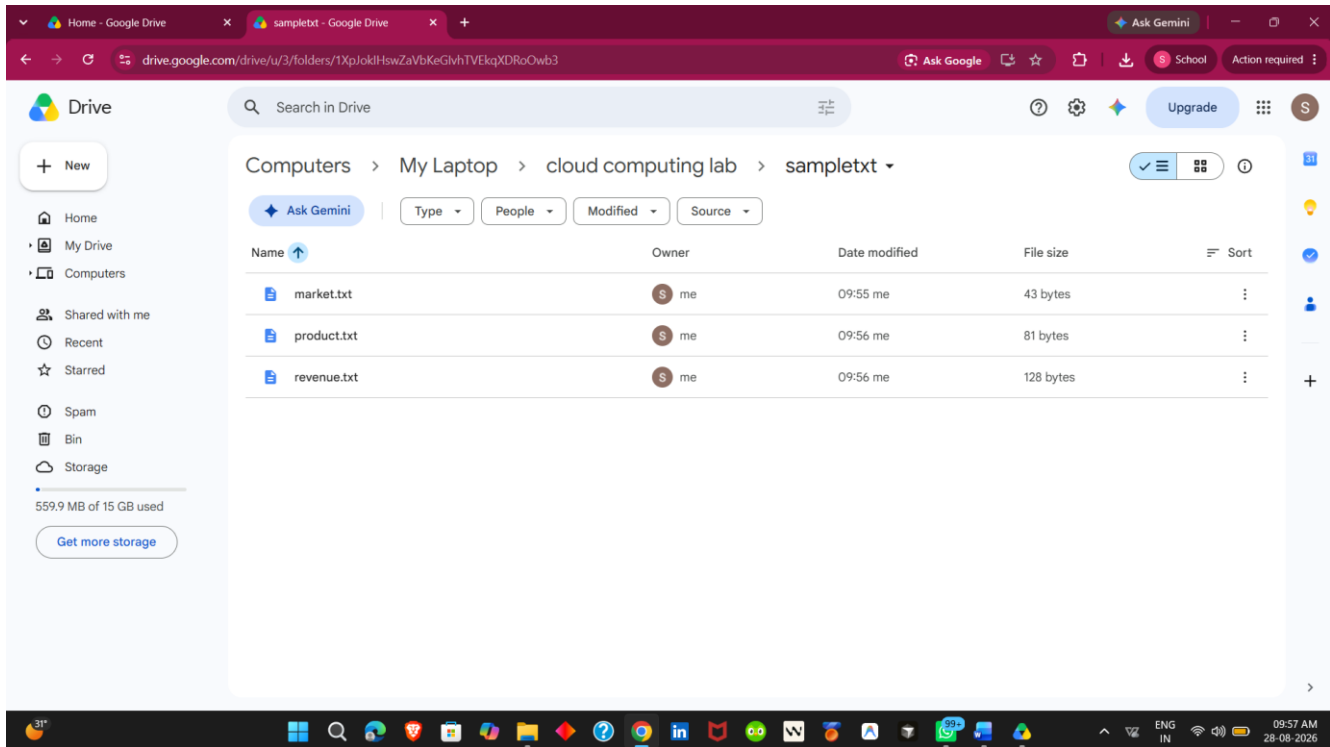


8. Verified in Google Drive that the synchronized OfficeData folder contained Market.txt, Revenue.txt, and Products.txt.

9. Opened the local Student.txt file and added the line: Cloud Backup Test - Successfully Synchronized. Saved the file.

10. Waited for synchronization and verified that Google Drive showed the file as successfully uploaded / up to date.

11. Refreshed Google Drive in the browser, opened Student.txt, and verified that the added test line was present.



### Observation:

The OfficeData folder and its three text files were successfully synchronized with Google Drive. After modifying Student.txt locally, the same change appeared in the cloud copy after synchronization.

### Result:

The cloud backup setup was successfully completed using Google Drive. Local files were migrated to cloud storage, automatic synchronization was enabled, and the update

made to Student.txt was successfully reflected in Google Drive. Thus, an effective cloud backup mechanism was demonstrated.

|                                   |                                                    |
|-----------------------------------|----------------------------------------------------|
| <b>Exp No:6</b>                   | <b>CDN Service Selection and CDN Configuration</b> |
| <b>Date:</b><br><b>21/08/2026</b> |                                                    |

**Aim:**

To select a Content Delivery Network (CDN) provider, deploy a simple web application using Cloudflare, access the application through Cloudflare's edge network, and verify the HTTP response and Cloudflare-specific headers using browser Developer Tools.

**Software Used:**

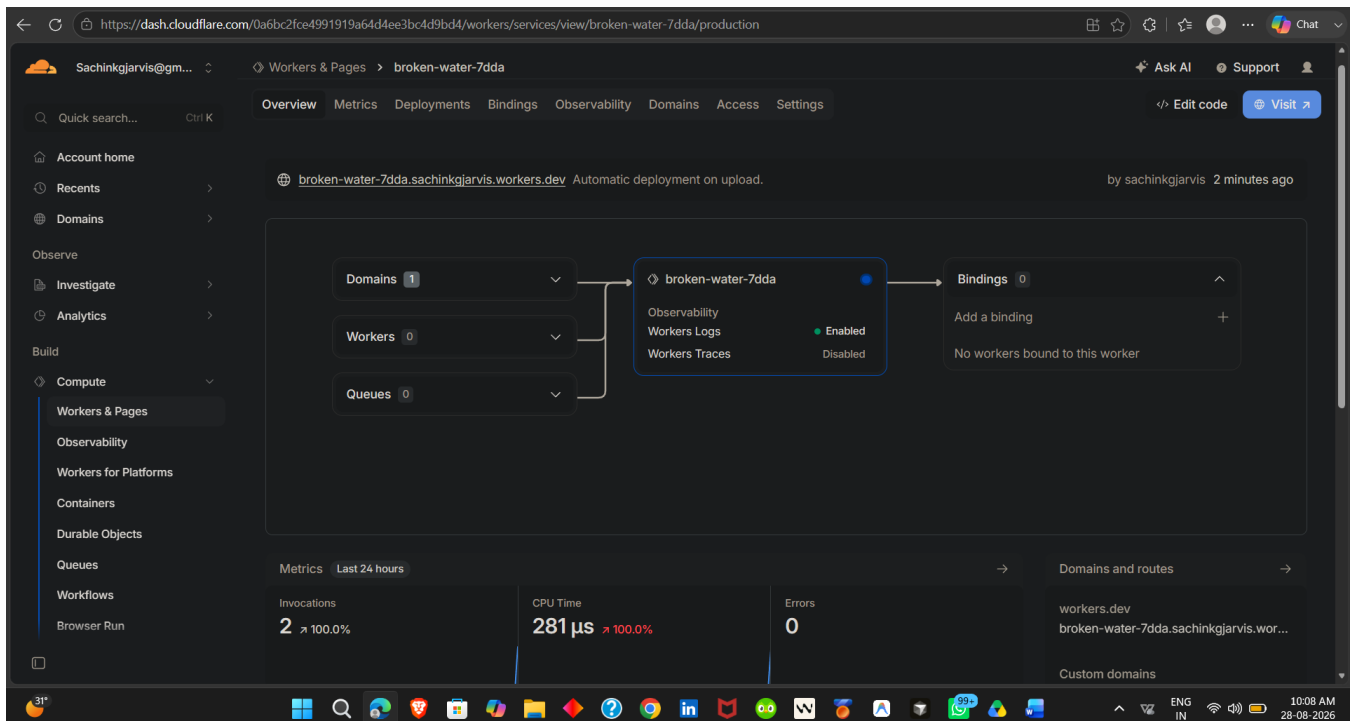
- Cloudflare Workers
- Google Chrome Web Browser
- Internet Connection

**CDN Provider Selection:**

Cloudflare was selected as the CDN/edge provider for this experiment. The working guide considered Cloudflare, Amazon CloudFront, and Akamai, and selected Cloudflare because it was the easiest option for the practical experiment.

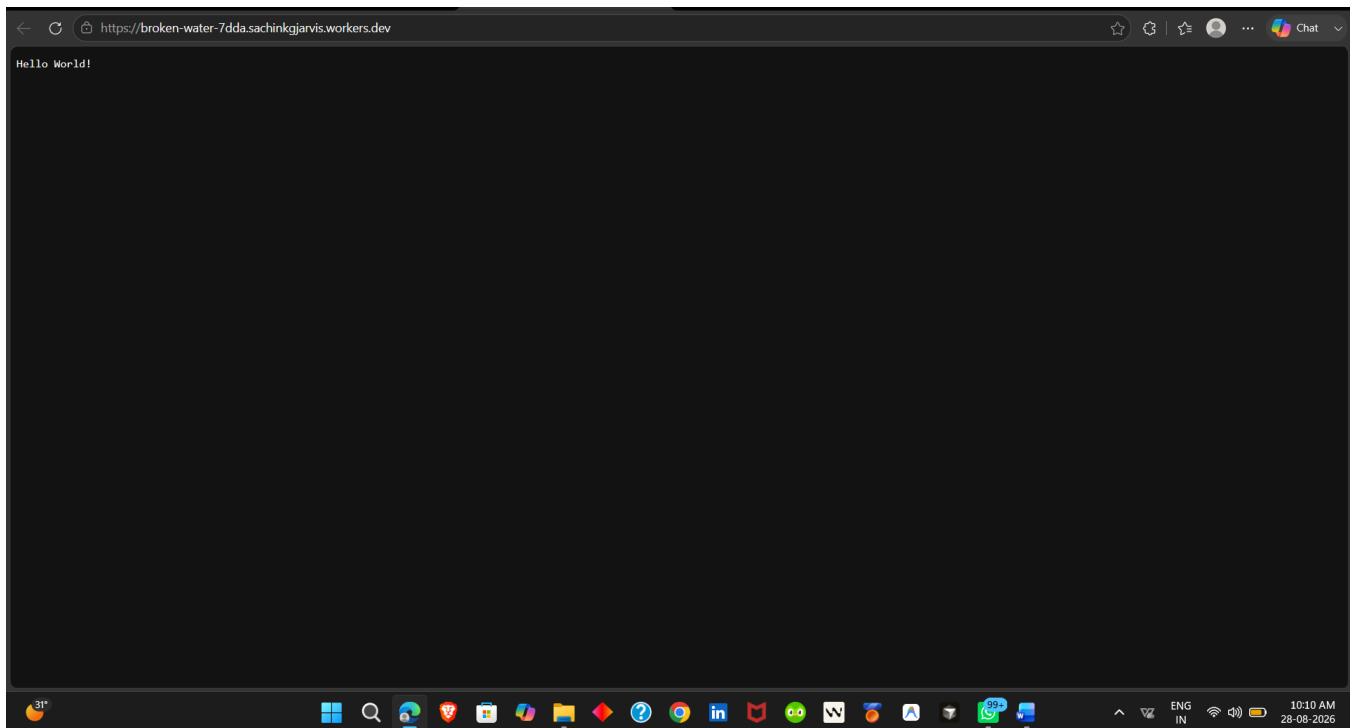
**Procedure:**

1. Created/logged into a Cloudflare account and opened the Cloudflare dashboard without changing any DNS or nameserver settings.
2. Selected Create app from the Cloudflare dashboard.
3. Selected Start with Hello World! as the application option.
4. Kept the automatically generated Worker name and the default Worker code unchanged.
5. Clicked Deploy and successfully deployed the Worker, obtaining a public workers.dev URL.

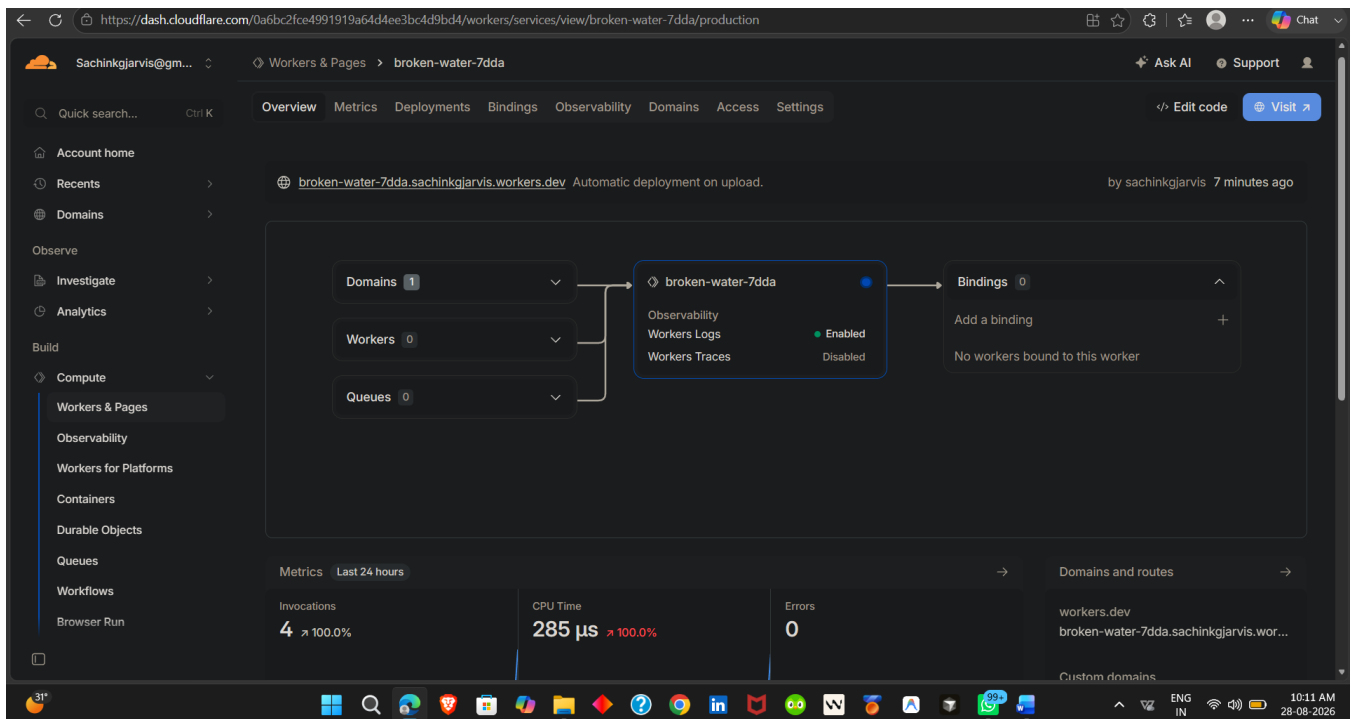


Cloudflare Worker successfully deployed with a workers.dev URL.

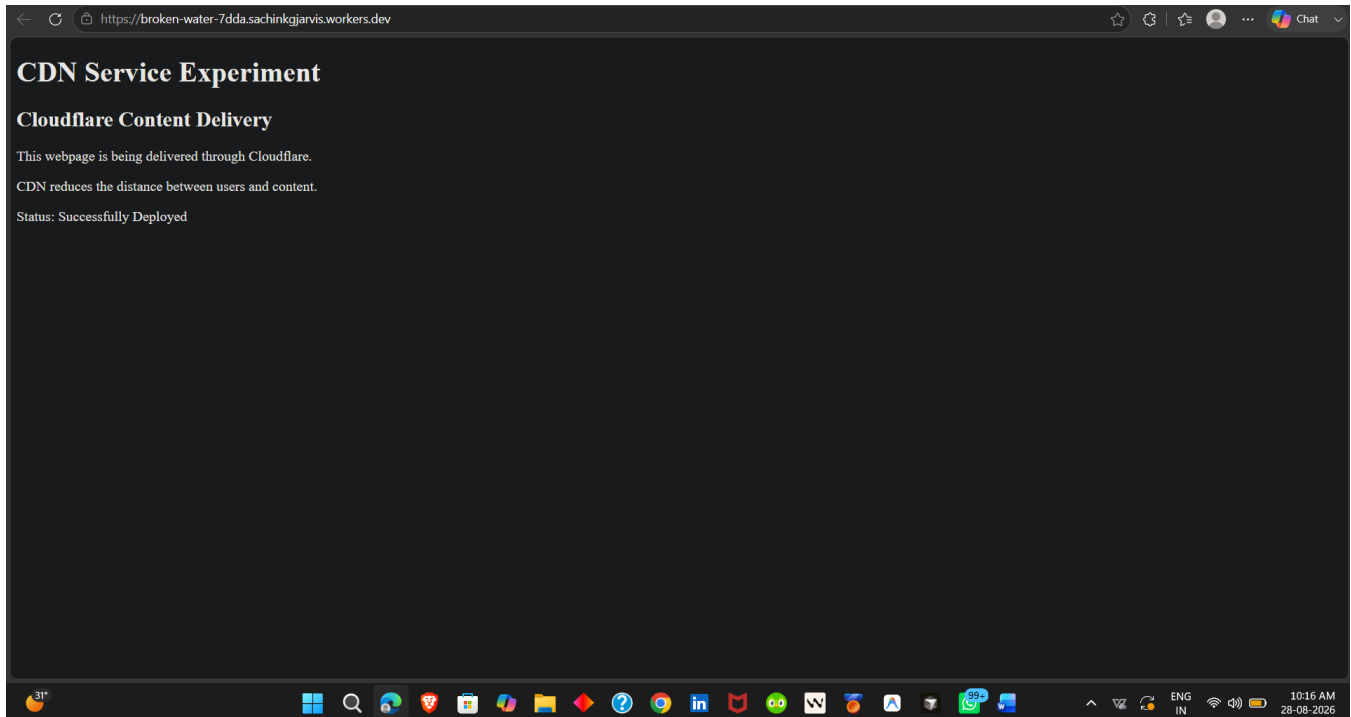
6. Clicked Visit to open the deployed Worker and verified that the page displayed Hello World!.
7. Returned to the Cloudflare dashboard and checked the Worker metrics. After the visit, the invocation count increased to 1 while the error count remained 0.
8. Opened Edit code and replaced the Hello World response with an HTML page titled CDN Service Experiment containing Cloudflare content-delivery information.
9. Deployed the updated Worker successfully.
10. Clicked Visit again and verified the CDN demonstration webpage with the messages CDN Service Experiment, Cloudflare Content Delivery, This webpage is being delivered through Cloudflare, CDN reduces the distance between users and content, and Status: Successfully Deployed.



Worker output displaying Hello World!

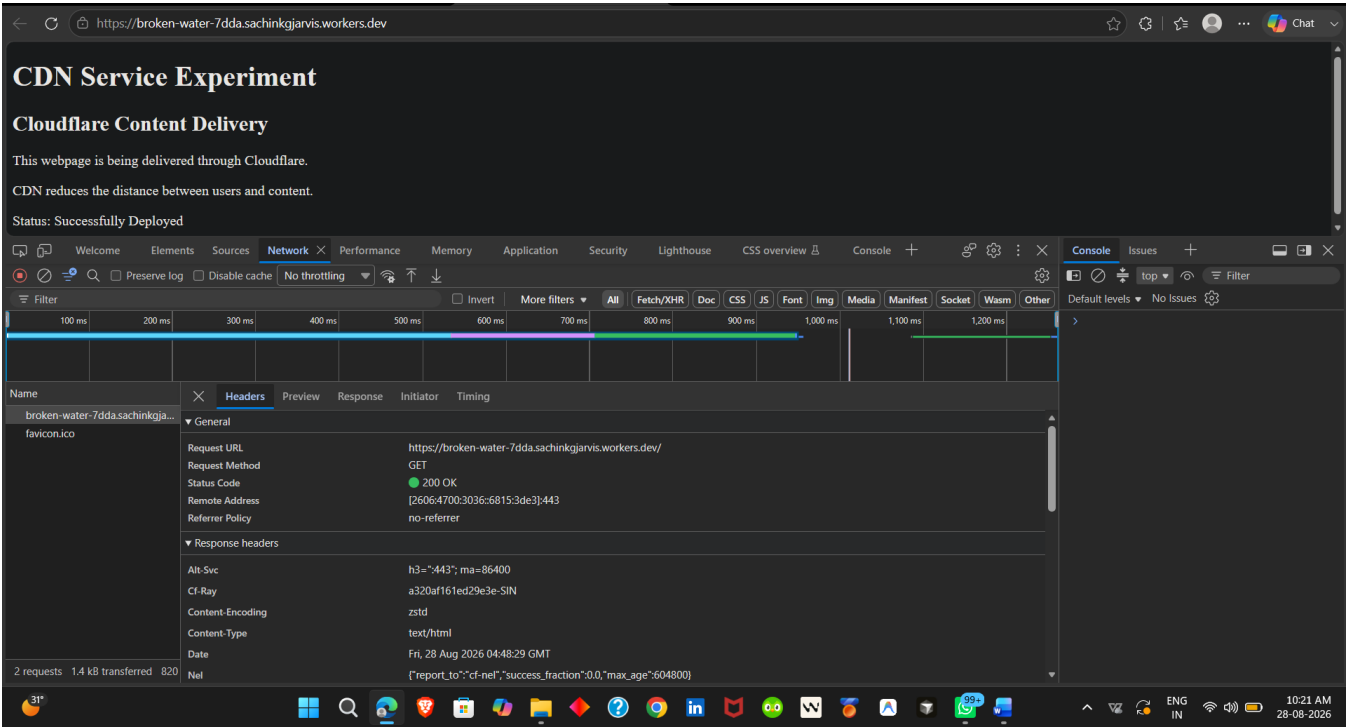


Cloudflare Metrics showing 4 invocation and 0 errors.

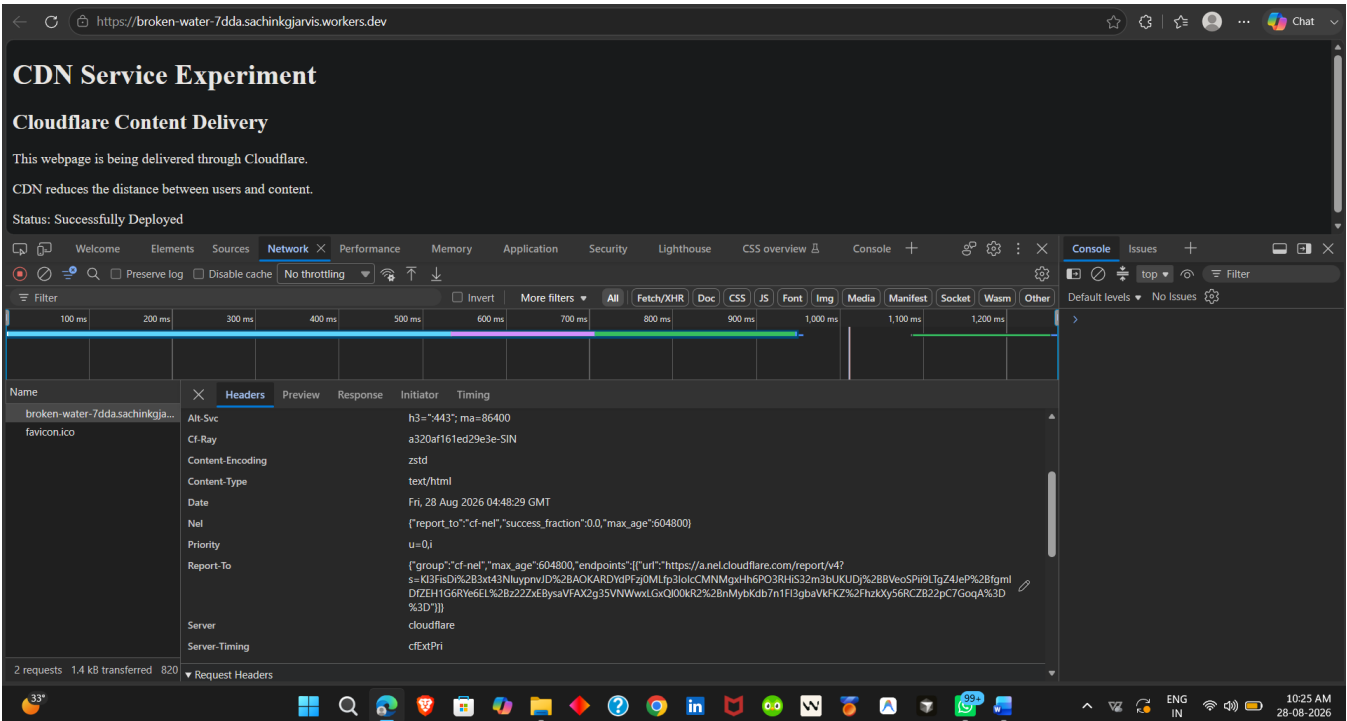


CDN Service Experiment webpage delivered through Cloudflare.

11. Opened Chrome Developer Tools using F12 and selected the Network tab.
12. Reloaded the CDN demonstration webpage and selected the request to the workers.dev address.
13. Verified the request method as GET and the HTTP status code as 200 OK.
14. Checked the response headers and verified Content-Type: text/html and the Cloudflare-specific CF-Ray header.
15. Scrolled further in the Response Headers section and verified Server: cloudflare, confirming that the response was handled by Cloudflare.



Developer Tools Network Headers showing GET, 200 OK, CF-Ray and Content-Type.



Response Headers showing Server: cloudflare and CF-Ray.

**Result:**

A Cloudflare-based edge content delivery experiment was successfully implemented. A Cloudflare Worker was deployed and accessed through a public workers.dev URL. The browser successfully received the CDN demonstration webpage with HTTP status 200 OK. The response headers included Cloudflare-specific information such as Server:

cloudflare and CF-Ray, confirming that the request was processed through Cloudflare's network.